Name of the Student	**D.N Shirshika**
Reg. No	**192425396**
GitHub Link	**https://github.com/shirshika572/Mla0403.git**

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning **COURSE CODE:MLA0403**

COURSE OUTCOME COVERED

CO 2: Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches.(BTL-4)

CO Weightage : 15%

Assessment Tool Weightage: 40%

Duration: 90 minutes

Marks: 25

Code of Conduct:

I ____D.N Shirshika(192425396)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Name of the student:** D.N Shirshika

Scenario-Based Questions

CO-AT2

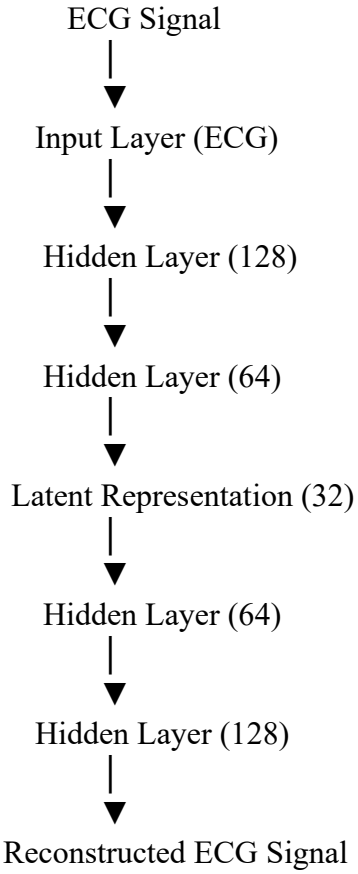
QUESTIONS: 05 **Total Marks:25**

1. **Unsupervised Training of Neural Networks, Auto Encoders**
   A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

   Introduction
   Electrocardiogram (ECG) signals are one of the most important diagnostic tools used by hospitals to monitor the electrical activity of the human heart. Doctors use ECG recordings to identify several cardiac abnormalities such as arrhythmia, atrial fibrillation, myocardial infarction, and other heart-related disorders. Modern hospitals generate an enormous amount of ECG data every day through continuous patient monitoring systems, wearable devices, and routine medical examinations. As the number of patients increases, manually analyzing every ECG record becomes difficult, time-consuming, and prone to human error.
   In many hospitals, ECG datasets are collected without labels because assigning labels requires expert cardiologists, making the process expensive and time-consuming. Since the available data is largely **unlabeled**, supervised learning methods cannot be applied effectively. Therefore, an **unsupervised deep learning technique** is required to automatically learn the underlying patterns of normal ECG signals and identify abnormal cases.
   Among the deep learning techniques included in the syllabus, the **Autoencoder** is the most suitable solution. Autoencoders are neural networks specifically designed to learn meaningful representations from unlabeled data by reconstructing their inputs. They can recognize normal ECG patterns during training and detect abnormal patterns by measuring reconstruction error.

   Why Autoencoders are Recommended
   An Autoencoder is a type of artificial neural network trained without requiring labeled data. Instead of predicting predefined classes, it learns how to compress the input data into a compact representation and then reconstruct it as accurately as possible.
   During training, the model is exposed to a large collection of ECG signals. As training progresses, the encoder learns the essential characteristics of normal heart activity, while the decoder learns to reconstruct the original ECG waveform from the compressed representation. Because the network becomes highly efficient at reconstructing normal ECG signals, it performs poorly when an abnormal signal is presented. The reconstruction error becomes significantly higher, allowing abnormal ECG records to be identified automatically.
   This makes Autoencoders highly suitable for anomaly detection problems in healthcare where labeled abnormal samples are limited.

Architecture of an Autoencoder



The Autoencoder consists of three major components:

1. Input Layer

The input layer receives the ECG signal after preprocessing. Each ECG recording consists of multiple numerical values representing the electrical activity of the heart over time.

2. Encoder

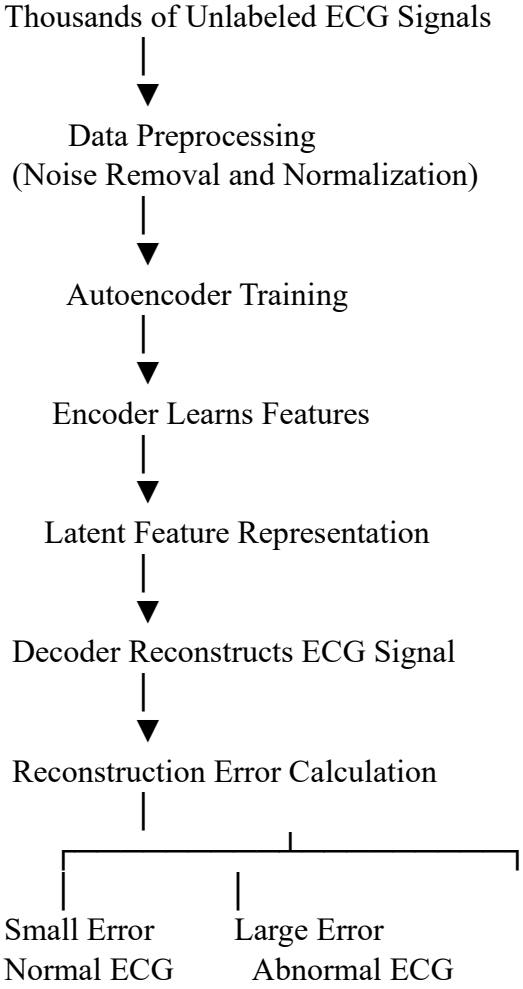
The encoder gradually reduces the dimensionality of the ECG signal. Instead of storing every data point, it learns only the most important features that describe the heartbeat. This compressed representation is called the **latent representation** or **feature vector**.

For example, an ECG signal containing 1000 numerical values may be compressed into only 32 important features without losing significant information.

3. Decoder

The decoder reconstructs the original ECG signal from the compressed representation. Its objective is to generate an output that closely matches the original input. During training, the network continuously adjusts its weights to minimize the reconstruction error between the original ECG signal and the reconstructed signal

Workflow of ECG Anomaly Detection



Step-by-Step Working Process

The first step is **data collection**, where thousands of ECG signals are gathered from hospital monitoring systems. Since these records are unlabeled, no information is available regarding whether they are normal or abnormal.

Next, the collected ECG signals undergo **data preprocessing**. Noise caused by electrical interference, patient movement, or sensor errors is removed using filtering techniques. The signals are then normalized so that all ECG recordings have a consistent range of values.

After preprocessing, the ECG signals are fed into the Autoencoder. During training, the encoder compresses each ECG signal into a lower-dimensional representation while preserving its important characteristics. This compressed representation contains meaningful information about the heartbeat, including rhythm, waveform shape, and timing relationships.

The decoder reconstructs the ECG signal from the compressed representation. The reconstructed signal is compared with the original ECG signal, and the difference between them is measured using a reconstruction error function such as Mean Squared Error (MSE).

The network continues training until the reconstruction error becomes very small for normal ECG signals.

When a new ECG signal is received, the trained Autoencoder attempts to reconstruct it. If the ECG pattern is normal, the reconstruction closely matches the input, resulting in a low reconstruction error. If the ECG contains abnormal characteristics, such as irregular heartbeats or unusual waveforms, the Autoencoder fails to reconstruct it accurately. Consequently, the reconstruction error becomes significantly larger, indicating the presence of an abnormal ECG.

Advantages of Using Autoencoders

Autoencoders provide several advantages for ECG analysis in hospitals. They eliminate the need for expensive manual labeling because they can learn directly from unlabeled ECG data. They automatically discover hidden patterns that may not be easily recognized by traditional machine learning algorithms. Since the model compresses the input into meaningful representations, it also performs dimensionality reduction while preserving important diagnostic information.
Another major advantage is the ability to detect previously unseen abnormalities.

Applications

Autoencoder-based ECG analysis has numerous applications in healthcare. It is widely used in intensive care units for continuous patient monitoring, wearable health devices for remote cardiac monitoring, emergency departments for rapid diagnosis, and telemedicine systems where ECG data is transmitted over long distances for automated analysis. It also supports predictive healthcare by identifying subtle changes in heart activity before serious complications occur

Conclusion

In conclusion, the **Autoencoder** is the most appropriate deep learning technique for identifying abnormal ECG patterns from thousands of unlabeled hospital records. Its unsupervised learning capability enables it to discover hidden patterns in ECG signals without requiring manually labeled data. By learning accurate representations of normal heart activity and detecting abnormalities through reconstruction error, Autoencoders provide an efficient, reliable, and scalable solution for automated ECG analysis. Their ability to perform anomaly detection, representation learning, and dimensionality reduction makes them an essential tool in modern healthcare systems, improving diagnostic accuracy while reducing the workload of medical professionals.

2. **Auto Encoders, Representation Learning**
   A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

   Introduction
   In today's retail industry, businesses collect massive amounts of customer data through online shopping platforms, supermarkets, mobile applications, and loyalty programs. Every purchase made by a customer generates valuable information such as the products purchased, quantity, purchase frequency, payment method, discount usage, shopping time, spending amount, and product categories. As the number of customers increases, the dataset becomes extremely large and complex. Such datasets often contain hundreds or even thousands of features describing customer purchasing behavior.
   Among the deep learning techniques studied in the syllabus, the **Autoencoder** is one of the most effective methods for dimensionality reduction. It performs **representation learning**, where the neural network automatically learns compact and meaningful representations of customer data without requiring labeled examples. These compressed representations can then be used as input for clustering algorithms such as K-Means to obtain accurate customer segments.
   \
   What is an Autoencoder?
   An **Autoencoder** is an unsupervised artificial neural network designed to learn efficient representations of input data. Unlike traditional neural networks that classify data into predefined categories, an Autoencoder learns by attempting to reconstruct its own input.
   The network compresses the original high-dimensional data into a smaller set of meaningful features called the **latent representation** or **bottleneck layer**. From this compressed representation, it reconstructs the original data as accurately as possible.
   During this process, the Autoencoder automatically discovers the most important patterns and relationships present in customer purchase data while eliminating redundant information.

   Need for Dimensionality Reduction
   Suppose a retail company stores the following information for every customer:

- Customer ID
- Age
- Gender
- Location
- Income
- Occupation
- Number of purchases
- Total spending
- Preferred shopping time
- Preferred payment method
- Favorite product categories

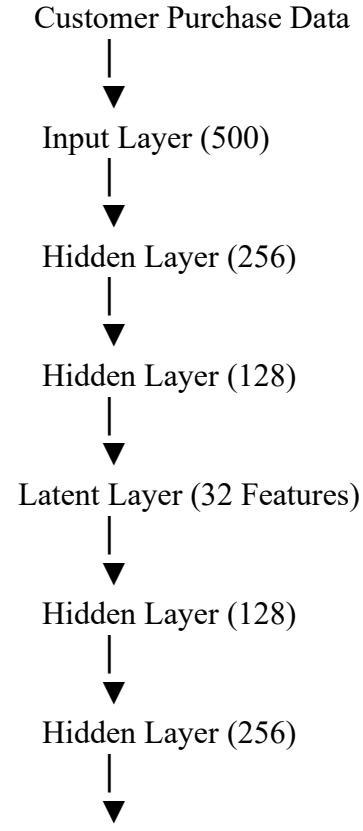
- Brand preferences
- Discount usage
- Seasonal shopping habits
- Purchase frequency
- Website browsing history
- Product ratings
- Return history
- Loyalty points
- Membership type

  A dataset may easily contain **500–1000 features** for each customer.

  Many of these variables carry similar information. Processing all these features directly increases computational cost and may reduce clustering performance due to the "curse of dimensionality."

  An Autoencoder solves this problem by learning a much smaller set of features that still captures the important characteristics of customer behavior.

Architecture of an Autoencoder



The architecture consists of three major components:

1. Input Layer

The input layer receives the complete customer purchase dataset. Each customer record contains numerous features describing purchasing habits and personal information.

2. Encoder

The encoder gradually compresses the input into fewer dimensions.

For example,

500 Features

↓

256 Features

↓

128 Features

↓

32 Features

During this compression process, the encoder removes unnecessary information while preserving the most useful characteristics.

3. Latent Representation

The compressed representation is called the **latent space**.

Instead of storing hundreds of variables, the Autoencoder stores only a few meaningful numerical values representing the customer's shopping behavior.

These latent features may capture information such as:

- Spending capacity
- Shopping frequency
- Product preference
- Brand loyalty
- Seasonal purchasing behavior
- Price sensitivity

Although these features are not manually defined, the Autoencoder automatically learns them during training.

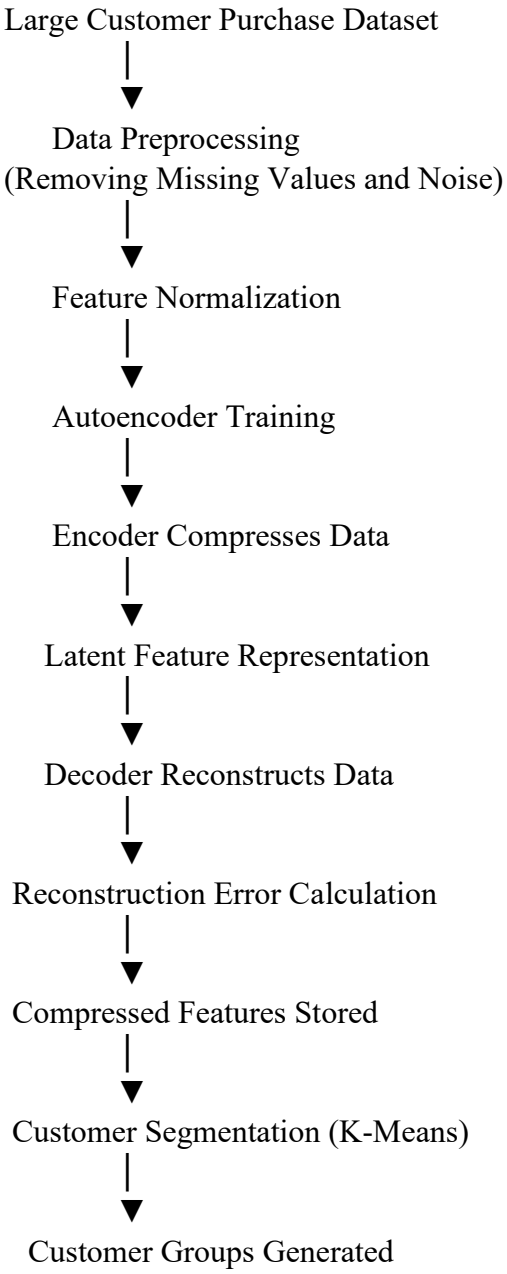
4. Decoder

The decoder reconstructs the original customer data from the latent representation.

Its objective is to reproduce the original input as accurately as possible.

The network continuously adjusts its parameters until the reconstruction error becomes very small.

Workflow of Customer Data Compression



Step-by-Step Working Process

The first stage involves collecting customer purchase data from various retail sources such as online shopping websites, supermarket billing systems, and loyalty card databases. Since this information often contains missing values, duplicate records, and inconsistent formats, data preprocessing is performed to clean the dataset.

After cleaning, all numerical values are normalized to ensure that different features have comparable scales. Normalization prevents features with larger numerical ranges from dominating the learning process.

The processed customer data is then supplied to the Autoencoder. During training, the encoder compresses the high-dimensional data into a low-dimensional latent representation. This compressed representation captures the essential purchasing characteristics of each customer while removing redundant information.

The decoder reconstructs the original customer data from the compressed representation. The difference between the reconstructed output and the original input is calculated using a reconstruction loss function such as Mean Squared Error (MSE). The network repeatedly updates its weights using backpropagation until the reconstruction error becomes minimal.

Once training is complete, the decoder is no longer required. Only the encoder is used to transform customer records into compact feature vectors.

These compressed feature vectors become the input for customer segmentation algorithms such as K-Means clustering. Since the latent representations contain only the most informative characteristics, customers with similar purchasing behavior naturally form clusters.

Representation Learning

Representation learning is the process of automatically discovering useful features from raw data instead of manually selecting them.

In traditional machine learning, data analysts spend considerable effort designing handcrafted features. For example, they may calculate average monthly spending, purchase frequency, or product diversity.

In contrast, an Autoencoder automatically learns hidden representations that may include:

- Overall purchasing power
- Preference for premium products
- Seasonal buying trends
- Interest in electronics or clothing
- Frequency of repeat purchases
- Brand loyalty
- Price sensitivity
- Shopping behavior consistency

These learned representations provide a more accurate description of customer behavior than manually designed features.

Customer Segmentation Using Latent Features

After dimensionality reduction, clustering algorithms group customers into meaningful segments.

For example:

Cluster 1 – Premium Customers

Customers who frequently purchase expensive products and have high annual spending.

Cluster 2 – Budget Customers

Customers who mainly buy discounted products and are highly price-sensitive.

Cluster 3 – Occasional Customers

Customers who shop only during festivals or seasonal sales.

Cluster 4 – Loyal Customers

Customers who repeatedly purchase products from the same brand and use loyalty rewards.

These customer groups help businesses develop effective marketing strategies.

Advantages of Using Autoencoders

Autoencoders provide several advantages for customer segmentation. They automatically reduce the dimensionality of complex datasets while preserving important information. Since the network learns directly from unlabeled data, manual feature engineering is not required. The compressed feature representations improve the performance of clustering algorithms and reduce computational cost. Autoencoders are also capable of removing noise from customer data, leading to more accurate segmentation results.

Another significant advantage is scalability. Retail companies with millions of customer records can efficiently process large datasets using Autoencoders without significant loss of information.

Limitations

Despite their advantages, Autoencoders also have certain limitations. They require a sufficiently large dataset to learn meaningful representations effectively. Training deep Autoencoders may require considerable computational resources and time. Selecting appropriate network architecture, latent dimension size, and hyperparameters is also important to achieve optimal performance. If the latent space is too small, useful information may be lost; if it is too large, dimensionality reduction becomes less effective.

Applications

Autoencoders are widely used in retail and e-commerce for customer segmentation, recommendation systems, market basket analysis, personalized product recommendations, fraud detection, sales forecasting, inventory management, and customer lifetime value prediction. They are also applied in banking, healthcare, manufacturing, and telecommunications for dimensionality reduction and representation learning.

Conclusion

In conclusion, the Autoencoder is an effective deep learning technique for reducing the dimensionality of customer purchase data before customer segmentation. By learning compact and meaningful representations of customer behavior, it removes redundant information while preserving the most significant purchasing characteristics. These learned representations improve clustering accuracy, reduce computational complexity, and help businesses identify meaningful customer groups for targeted marketing and personalized services. Through representation learning and unsupervised training, Autoencoders have become one of the most powerful tools for analyzing high-dimensional retail datasets.

3. **Width and Depth of Neural Networks, Activation Functions**

A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

Introduction

Facial recognition is one of the most widely used applications of deep learning in today's world. It is used in smartphones for secure unlocking, airports for passenger identification, banks for customer verification, educational institutions for attendance systems, and law enforcement agencies for surveillance. A facial recognition system works by analyzing facial images and identifying or verifying the identity of a person based on unique facial characteristics.

To overcome this limitation, the network architecture should be modified by increasing the **depth** of the neural network and selecting appropriate **activation functions**. A deeper neural network is capable of learning hierarchical features from facial images, while suitable activation functions improve the learning capability of the network by introducing non-linearity. These architectural improvements significantly enhance the accuracy, robustness, and generalization ability of facial recognition systems.

Understanding Shallow and Deep Neural Networks

A neural network consists of an input layer, one or more hidden layers, and an output layer. The number of hidden layers determines the **depth** of the network.

A **shallow neural network** generally contains only one or two hidden layers. Such networks are capable of solving simple classification problems but are not powerful enough to learn highly complex image features.

A **deep neural network**, on the other hand, contains many hidden layers. Each layer learns increasingly abstract features from the input image, allowing the network to recognize complex objects such as human faces with much greater accuracy.

Problems with a Shallow Neural Network

When a facial recognition system uses a shallow neural network, several problems arise. The network is unable to extract detailed facial features because it lacks sufficient layers to learn hierarchical representations. As a result, important characteristics such as eye shape, nose structure, facial contours, and expressions may not be captured effectively.

Another issue is poor generalization. The network may perform well on training images but fail to recognize faces under different lighting conditions, viewing angles, facial expressions, or partial occlusions such as glasses and masks. Shallow networks also have limited capacity to model the highly nonlinear relationships present in facial image data, resulting in lower recognition accuracy.

Recommended Architectural Changes

To improve the performance of the facial recognition system, the following architectural changes are recommended:

1. Increase the depth of the neural network.
2. Use Convolutional Neural Network (CNN) architecture.
3. Replace traditional activation functions with ReLU or Leaky ReLU.
4. Use Softmax activation in the output layer.
5. Include pooling layers to reduce feature dimensions and improve robustness.
6. Add Batch Normalization and Dropout to improve training stability and reduce overfitting.

These changes allow the network to learn more meaningful facial representations and improve recognition performance.

Role of Network Depth

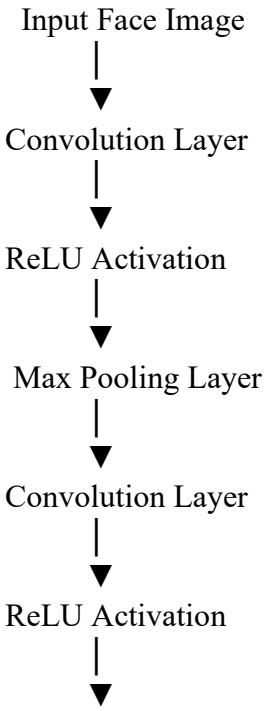
The depth of a neural network refers to the number of hidden layers between the input and output layers. Increasing the depth enables the network to learn progressively more complex features.

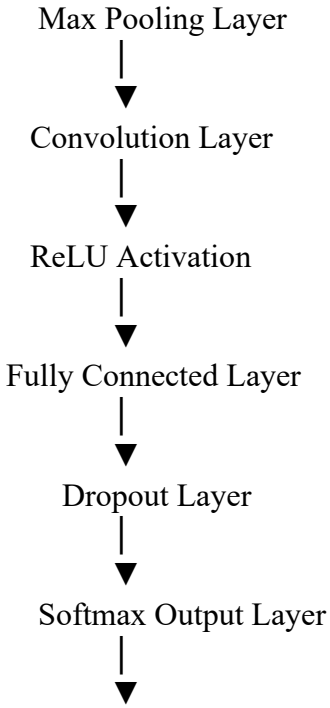
In facial recognition, different layers learn different types of information:

- The first layer learns simple edges and lines.
- The second layer identifies corners and textures.
- The third layer detects facial components such as eyes, nose, and mouth.
- The fourth layer combines these components to form complete facial structures.
- The deeper layers learn highly discriminative facial representations that uniquely identify individuals.

This hierarchical feature learning is the main reason why deep neural networks outperform shallow networks in image recognition tasks.

Architecture of the Improved Deep Neural Network





Importance of Activation Functions

Activation functions determine whether a neuron should be activated or not. They introduce non-linearity into the network, enabling it to learn complex relationships.

Without activation functions, a deep neural network behaves like a simple linear model regardless of the number of layers.

Choosing the correct activation function is therefore essential for improving facial recognition performance.

ReLU (Rectified Linear Unit)

The most commonly used activation function in deep learning is the **Rectified Linear Unit (ReLU)**.

It is mathematically defined as:

$$f(x) = \max(0, x)$$

If the input value is positive, the output remains unchanged. If the input is negative, the output becomes zero.

Advantages of ReLU

ReLU accelerates the training process because it is computationally simple. It also helps solve the vanishing gradient problem, allowing deep neural networks to learn efficiently. Since only positive neurons remain active, the network becomes computationally efficient and converges faster.

For facial recognition, ReLU enables the network to learn complex facial features without slowing down the training process.

Leaky ReLU

One limitation of ReLU is the "dead neuron problem," where neurons receiving negative inputs stop learning permanently.

Leaky ReLU addresses this issue by allowing a small negative slope.

Its mathematical expression is:

$$f(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x < 0 \end{cases}$$

This ensures that neurons continue learning even when the input is negative.
Leaky ReLU is particularly useful in deeper networks where dead neurons can significantly reduce learning capacity.

Softmax Activation Function
The output layer of a facial recognition system typically contains one neuron for each known individual.
Softmax converts the output values into probabilities whose total equals one.
For example,

Person	**Probability**
Person A	0.05
Person B	0.90
Person C	0.03
Person D	0.02

The system predicts **Person B** because it has the highest probability.
Softmax is therefore the preferred activation function for multi-class facial recognition problems.

Feature Learning in Deep Networks
A deep neural network learns facial features gradually.
Input Face Image
↓
Layer 1 → Detects Edges
↓
Layer 2 → Detects Corners and Textures
↓
Layer 3 → Detects Eyes, Nose and Mouth
↓
Layer 4 → Learns Complete Face Structure
↓
Layer 5 → Identifies Individual Person

Applications

Deep facial recognition systems are widely used in smartphone face unlocking, biometric attendance systems, airport security, border control, banking authentication, online examination proctoring, criminal identification, surveillance systems, and healthcare patient verification.

Conclusion

A shallow neural network is insufficient for accurate facial recognition because it cannot capture the complex hierarchical features present in facial images. Increasing the network depth enables the model to learn progressively richer representations, from simple edges to complete facial identities. Using **ReLU or Leaky ReLU** in hidden layers improves learning efficiency and prevents vanishing gradient problems, while **Softmax** provides accurate multi-class classification in the output layer. Incorporating convolutional layers, pooling, Batch Normalization, and Dropout further enhances performance. Together, these architectural improvements produce a robust, accurate, and scalable facial recognition system capable of handling real-world variations in human faces.

4. **Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks**

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

Introduction

In today's digital world, online streaming platforms such as Netflix, Amazon Prime Video, Disney+, and other video-on-demand services contain millions of movies, television shows, documentaries, and web series. Every day, millions of users watch different types of content based on their interests, preferences, and viewing habits. Because of this enormous amount of available content, it becomes difficult for users to find movies that match their interests. To solve this problem, streaming services use **recommendation systems** that automatically suggest movies and TV shows that users are likely to enjoy.

A recommendation system analyzes the viewing history, ratings, watch duration, favorite genres, and user preferences to predict which movies should be recommended. However, in many practical situations, there are no labeled datasets indicating which movies should be recommended to each user. Instead, the system only has access to user viewing history and ratings. This makes **unsupervised learning** an ideal approach.

One of the most effective unsupervised deep learning techniques for recommendation systems is the **Restricted Boltzmann Machine (RBM)**. RBMs are capable of learning hidden relationships between users and movies without requiring labeled data. They automatically discover latent features such as user interests, preferred movie genres, and viewing patterns. These learned features enable the recommendation engine to predict movies that a user is likely to watch in the future.

What is a Restricted Boltzmann Machine (RBM)?

A Restricted Boltzmann Machine (RBM) is a type of **unsupervised artificial neural network** used for learning hidden representations of data. It belongs to the family of probabilistic graphical models and is widely used for dimensionality reduction, feature learning, collaborative filtering, and recommendation systems.

An RBM consists of two layers:

1. **Visible Layer**
2. **Hidden Layer**

Unlike traditional neural networks, RBMs do not have connections between neurons within the same layer. Connections exist only between the visible and hidden layers. This restriction simplifies the learning process and improves computational efficiency. The RBM learns the probability distribution of the input data and automatically discovers hidden patterns without requiring labeled examples.

Need for RBMs in Movie Recommendation

Suppose a streaming service has millions of users and thousands of movies.

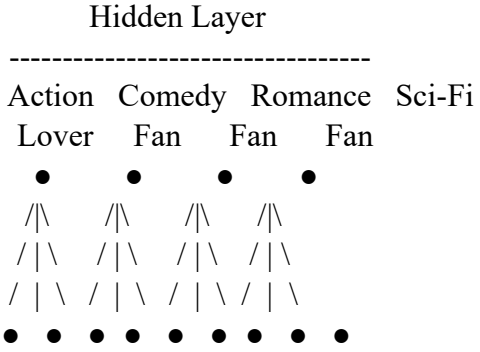
Each user watches only a small number of movies.

For example,

User	Movie A	Movie B	Movie C	Movie D	Movie E
User 1	✓	✓	✗	✓	✗
User 2	✗	✓	✓	✗	✓
User 3	✓	✗	✓	✓	✗

This data is called a **User-Movie Interaction Matrix**.
The objective is to predict which unseen movies each user is likely to enjoy.
Since no labels are available indicating the "correct" recommendations, supervised learning cannot be applied directly.
An RBM solves this problem by discovering hidden relationships among users and movies.

Architecture of Restricted Boltzmann Machine

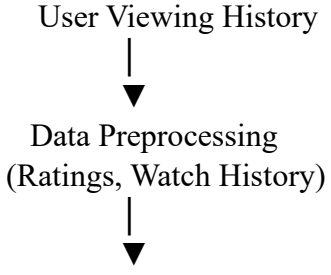


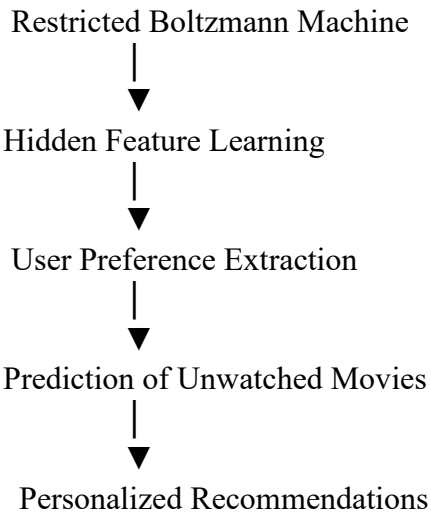
The **Visible Layer** represents the movies that users have watched or rated.
The **Hidden Layer** represents hidden user interests such as:

- Action movie lover
- Comedy fan
- Romantic movie fan
- Science fiction enthusiast
- Thriller lover
- Animation fan

The network automatically learns these hidden preferences during training.

Workflow of Movie Recommendation System





Step-by-Step Working Process

The first step is collecting user interaction data. Every movie watched by a user is stored in a database along with additional information such as ratings, watch duration, likes, and watch frequency.

After data collection, preprocessing is performed. Missing ratings are handled, duplicate entries are removed, and the user-movie interaction matrix is created.

This matrix is then provided as input to the Restricted Boltzmann Machine.

The visible layer receives information about movies watched by users.

During training, the hidden layer learns latent features representing user interests.

For example, if many users who watch superhero movies also watch science fiction movies, the RBM automatically discovers this relationship.

Similarly, if users who enjoy romantic comedies often watch family movies, the network learns this hidden pattern without any manual programming.

After multiple training iterations, the RBM becomes capable of predicting missing values in the user-movie matrix.

These predicted values represent the probability that a user will enjoy a particular movie.

Movies with the highest probabilities are recommended to the user.

Learning Hidden Features

One of the biggest strengths of RBMs is their ability to learn **latent features**.

Instead of manually defining user interests, the network automatically discovers them.

Examples include:

- Preference for action movies
- Interest in science fiction
- Love for animated movies
- Preference for romantic dramas
- Interest in thriller movies
- Family entertainment preference
- Regional language preference
- Watching movies during weekends

These hidden features are learned directly from user behavior.

Collaborative Filtering using RBM

RBMs are commonly used for **Collaborative Filtering**, one of the most popular recommendation techniques.

The basic idea is:

Users with similar viewing histories usually have similar preferences.

For example,

Suppose User A and User B have watched the following movies:

User	Action	Comedy	Sci-Fi
User A	✓	✓	✓
User B	✓	✓	?

Since both users have similar viewing patterns, the RBM predicts that User B is also likely to enjoy the Science Fiction movie.

Thus, the recommendation engine suggests the movie automatically.

Training of RBM

Training an RBM involves adjusting the weights between visible and hidden neurons.

The most common learning algorithm used is **Contrastive Divergence (CD)**.

The training process consists of:

1. Feed the user-movie interaction matrix to the visible layer.
2. Calculate hidden neuron activations.
3. Reconstruct the visible layer.
4. Compare reconstructed data with original data.
5. Update weights.
6. Repeat until reconstruction error becomes very small.

This iterative learning enables the RBM to accurately model user preferences.

Advantages of RBMs

Restricted Boltzmann Machines offer several advantages for recommendation systems. Since they are unsupervised learning models, they do not require labeled datasets, making them suitable for real-world applications where labeled recommendation data is unavailable. RBMs automatically learn hidden user preferences, reducing the need for manual feature engineering. They are highly effective in handling sparse user-movie interaction matrices, where most users have watched only a small subset of available movies.

Another major advantage is that RBMs can discover complex relationships between users and movies that may not be obvious through traditional recommendation methods. They also improve recommendation accuracy by identifying latent similarities among users with similar viewing habits.

Limitations

Despite their advantages, RBMs have certain limitations. Training deep RBMs can be computationally expensive, especially when the number of users and movies is very large. They require careful tuning of hyperparameters such as learning rate, number of

hidden neurons, and training epochs. RBMs may also struggle with completely new users who have no viewing history, a problem commonly known as the **cold-start problem**.

Applications
Restricted Boltzmann Machines are widely used in various recommendation systems, including movie recommendation on streaming platforms, music recommendation services, online shopping websites, news article recommendations, social media content suggestions, online advertising, e-learning platforms, and personalized healthcare recommendations. They are also used for feature extraction, dimensionality reduction, and representation learning in deep learning applications.

Real-World Example
Consider a streaming platform where a user has watched several superhero and science fiction movies, including *Avengers*, *Iron Man*, and *Guardians of the Galaxy*. The RBM analyzes the viewing patterns of millions of users and discovers that people who enjoy these movies often watch *Doctor Strange* and *Black Panther*. Even though the user has never watched these movies, the RBM predicts a high probability that they will enjoy them and recommends them on the homepage. This personalized recommendation enhances the user experience and increases customer satisfaction.

Conclusion
In conclusion, the **Restricted Boltzmann Machine (RBM)** is a powerful unsupervised deep learning model for building movie recommendation systems. By learning hidden relationships between users and movies without requiring labeled data, RBMs can accurately identify user preferences and recommend relevant content. Their ability to perform collaborative filtering, representation learning, and feature extraction makes them highly effective for modern streaming platforms. Although they require computational resources for training, their capability to provide personalized movie recommendations significantly improves user engagement, customer satisfaction, and platform usage. Therefore, RBMs remain one of the most important deep learning techniques for developing intelligent recommendation engines in the entertainment industry.

5. Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

Introduction

In modern manufacturing industries, maintaining high product quality is one of the most important objectives. Every manufacturing company aims to produce products that meet quality standards while minimizing defects, production costs, and wastage. To achieve this goal, factories use numerous sensors installed on production machines to continuously monitor the manufacturing process. These sensors collect information such as temperature, pressure, vibration, humidity, voltage, current, machine speed, and product dimensions every second.

One of the most effective approaches is to combine representation learning with deep neural networks. Representation learning enables the model to automatically learn meaningful features from raw sensor data instead of relying on manually engineered features. An Autoencoder can first learn compact representations of normal operating conditions, and a Deep Neural Network (DNN) can then use these learned features to classify products as normal or defective. This intelligent system improves quality control, reduces production losses, and enables predictive maintenance.

Problem Statement

A manufacturing company collects sensor readings every second from machines involved in the production process. The sensors measure several parameters, including:

- Temperature
- Pressure
- Vibration
- Voltage
- Current
- Machine Speed
- Humidity
- Product Weight
- Product Thickness
- Energy Consumption

Representation Learning

Representation learning is a technique in which a neural network automatically learns meaningful features from raw input data. Instead of manually selecting variables that may indicate product quality, the model discovers important patterns directly from sensor readings.

For example, instead of explicitly programming rules such as:

- Temperature greater than 100°C indicates a defect.

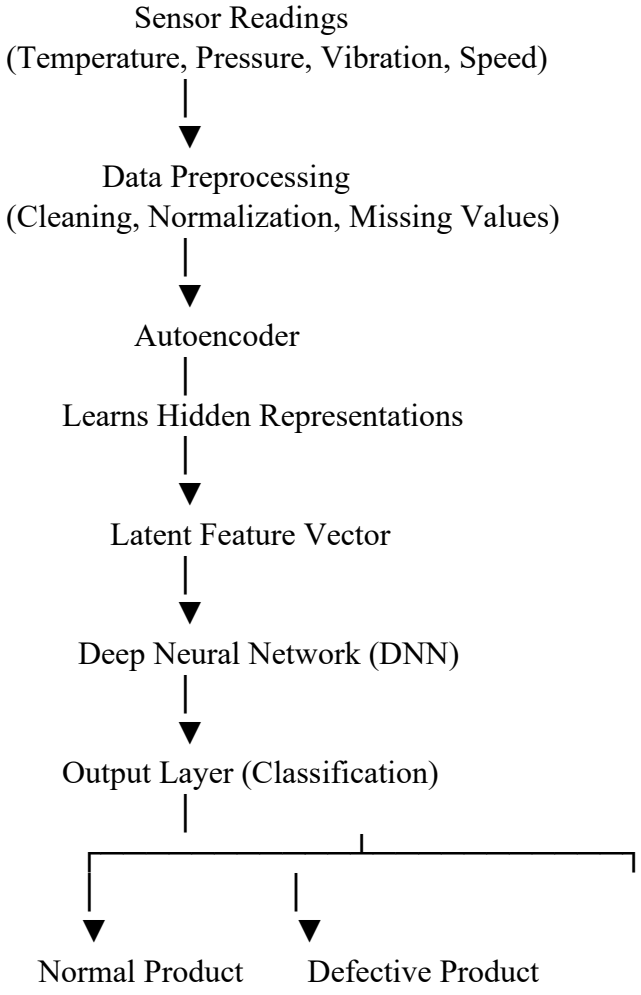
- High vibration indicates machine failure.
  The neural network automatically learns combinations of sensor values that correspond to normal and defective products.
  These learned representations provide a more accurate understanding of machine behavior and product quality.

Proposed Deep Learning Architecture

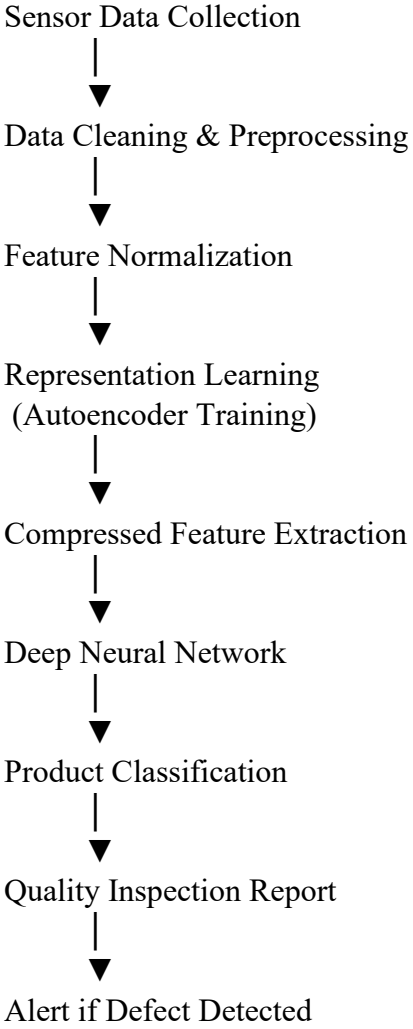
The proposed system consists of three major components:

1. Data Preprocessing
2. Representation Learning using an Autoencoder
3. Product Classification using a Deep Neural Network

Architecture Diagram



Workflow of the Proposed System



Step-by-Step Working Process

The first step is sensor data collection. Every second, sensors installed on manufacturing equipment record measurements such as temperature, pressure, vibration, current, voltage, humidity, and machine speed. These readings are stored in a centralized database.

The collected data is then preprocessed to remove missing values, eliminate noisy measurements, and normalize the sensor values. Data normalization ensures that all sensor readings are within a comparable range, improving the stability of neural network training.

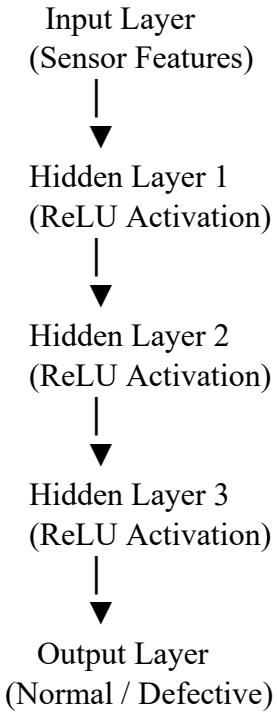
Next, the preprocessed data is provided as input to an Autoencoder. The encoder compresses the high-dimensional sensor data into a smaller latent representation while preserving the most significant information. This compressed representation captures hidden relationships among sensor variables that describe normal machine operation.

The decoder reconstructs the original sensor readings from the latent representation. During training, the Autoencoder minimizes the reconstruction error between the original and reconstructed data. After successful training, the encoder alone is used to generate compact feature vectors for new sensor readings.

These learned feature vectors are then passed to a Deep Neural Network (DNN). The DNN contains multiple hidden layers with nonlinear activation functions such as ReLU. The network analyzes the compressed features and predicts whether the product is normal or defective.

Finally, the output layer produces the classification result. If the product is predicted to be defective, an alert is sent to the quality control department or the production line is temporarily stopped to prevent further defective products from being manufactured.

Deep Neural Network Structure



Role of Representation Learning

Representation learning plays a crucial role in this solution because raw sensor readings often contain redundant and noisy information. The Autoencoder automatically discovers meaningful patterns that represent machine behavior.

Some examples of learned representations include:

- Stable operating conditions
- Abnormal vibration patterns
- Temperature-pressure relationships
- Energy consumption trends
- Machine wear characteristics
- Production cycle variations
- Hidden fault signatures

Limitations

Although the proposed solution provides significant benefits, it also has certain limitations. Training deep learning models requires a large amount of historical sensor data and powerful computational resources such as GPUs. Hyperparameter tuning,

including the number of hidden layers, learning rate, and latent feature size, is necessary to achieve optimal performance. Additionally, if sensor data quality is poor or contains excessive noise, the performance of the model may decrease.

Applications

Deep learning-based defect detection systems are widely used in various industries, including automobile manufacturing, electronics assembly, semiconductor production, pharmaceutical manufacturing, food processing, textile production, steel manufacturing, aerospace engineering, and smart factories. These systems are also used for predictive maintenance, machine health monitoring, industrial automation, and quality assurance.

Conclusion

In conclusion, a deep learning solution that combines representation learning with Autoencoders and Deep Neural Networks provides an effective approach for detecting defective products from continuously collected sensor readings. Representation learning automatically extracts meaningful features from raw sensor data, while the neural network accurately classifies products as normal or defective. The system supports real-time quality monitoring, reduces production losses, improves manufacturing efficiency, and enables predictive maintenance. Owing to its ability to process large volumes of industrial sensor data and identify complex defect patterns, this approach has become an essential technology in modern smart manufacturing and Industry 4.0 environments**.**

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand